HARI 2

Panduan lab hari 2

Building Modern Web Application

Praktik Hari 2 melanjutkan proyek Express dan EJS dengan validasi formulir, basis data PostgreSQL di Supabase, Prisma ORM, penyimpanan data barang, dan komponen partial yang dipakai bersama.

Urutan praktik

Lab	Topik praktik	Halaman
01	Validasi formulir	2
02	Menyiapkan PostgreSQL di Supabase	4
03	Menghubungkan aplikasi dengan Prisma ORM	5
04	Operasi CRUD dasar	6
05	Komponen partial EJS	12

LAB 01

Validasi formulir

Tujuan

Menerima data melalui metode POST dan memeriksa nama serta format email di sisi server menggunakan express-validator.

Langkah praktik

1. Buka terminal pada proyek Hari 1, lalu instal pustaka validasi:

```
npm install express-validator
```

2. Di app.js, tambahkan impor berikut di dekat impor Express. Letakkan konfigurasi pembacaan formulir setelah const app = express(); dan sebelum semua rute:

```
const { body, validationResult } = require('express-validator');

app.set('view engine', 'ejs');
app.use(express.urlencoded({ extended: true }));
```

Jika pengaturan view engine sudah ada dari Hari 1, gunakan pengaturan tersebut.

3. Buat views/form.ejs dan isi dengan kode berikut. Input email memakai tipe teks agar format yang salah tetap dapat dikirim untuk menguji validasi server:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Validasi formulir</title>
 <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body class="container mt-5">
 <h2>Form tambah data</h2>
 <% if (typeof errors !== 'undefined' && errors.length) { %>
  <div class="alert alert-danger" role="alert">
   <ul class="mb-0">
    <% errors.forEach(err => { %>
     <li><%= err.msg %></li>
    <% }); %>
   </ul>
  </div>
 <% } %>
 <form action="/submit-form" method="POST">
  <div class="mb-3">
   <label for="nama" class="form-label">Nama</label>
   <input id="nama" type="text" name="nama" class="form-control">
  </div>
  <div class="mb-3">
   <label for="email" class="form-label">Email</label>
   <input id="email" type="text" name="email" class="form-control">
  </div>
  <button type="submit" class="btn btn-primary">Kirim</button>
 </form>
</body>
</html>
```

Rute POST dan pengujian validasi

4. Tambahkan rute GET berikut di app.js untuk menampilkan formulir:

```
app.get('/form', (req, res) => {
 res.render('form');
});
```

5. Tambahkan rute POST beserta aturan validasinya setelah rute GET. trim() menghapus spasi di tepi nama, sehingga nama berisi spasi saja tetap ditolak:

```
app.post(
 "/submit-form",
 [
  body("nama").trim().notEmpty().withMessage("Nama tidak boleh kosong"),
  body("email").trim().isEmail().withMessage("Format email tidak valid"),
 ],
 (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
   return res.status(422).render("form", {
    errors: errors.array(),
   });
  }
  res.render("form-success", {
   nama: req.body.nama,
   email: req.body.email,
  });
 },
);
```

6. Buat halaman “form-success.ejs” pada views.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sukses</title>
  <link rel="stylesheet" href="/css/bootstrap.min.css">
</head>
<body>
  <h1>Formulir berhasil dikirim!</h1>
  <p>Nama: <%= nama %></p>
  <p>Email: <%= email %></p>
  <a href="/form" class="btn btn-primary">Kembali ke Form</a>
</body>
</html>
```

7. Simpan berkas, jalankan ulang server, lalu buka http://localhost:3000/form.

Hasil pembelajaran

Rute GET menampilkan formulir. Rute POST membaca req.body, memvalidasi isinya, lalu menampilkan pesan kesalahan atau hasil yang berhasil diproses. Data formulir ini belum disimpan ke basis data.

LAB 02

Menyiapkan PostgreSQL di Supabase

Tujuan

Membuat basis data PostgreSQL di Supabase dan menyimpan alamat koneksinya sebagai variabel lingkungan pada proyek Node.js.

Langkah praktik

1. Buka https://supabase.com/dashboard dan masuk ke akun Anda. Pilih organisasi, lalu New project.
2. Isi nama proyek, kata sandi database, dan region terdekat yang tersedia. Pilih paket Free bila tersedia, lalu tunggu proyek siap.

Periksa kuota dan kebijakan jeda pada dasbor. Untuk akses hanya melalui Prisma, nonaktifkan Data API pada pengaturan API proyek latihan.

3. Buka Connect, pilih Session pooler port 5432, lalu salin URI. Ganti kata sandi database; encode karakter khusus, misalnya @ menjadi %40. Tambahkan ?sslmode=require atau &sslmode=require jika URI sudah memiliki parameter. Project URL dan API key bukan URI database.

```
postgresql://postgres.iwkqbdmnrqhgtmiiwktp:Inipasword_123@aws-0-ap-southeast-
1.pooler.supabase.com:5432/postgres?sslmode=require
```

4. Kembali ke terminal proyek dan instal dotenv:

```
npm install dotenv
```

5. Buat .env sejajar dengan package.json. Isi PORT dan DATABASE_URL berikut, lalu ganti placeholder dengan URI Session pooler dari dasbor. Simpan URI sebagai satu baris; jangan menggandakan kunci:

```
PORT=3000
DATABASE_URL="GANTI_DENGAN_URI_SESSION_POOLER_SUPABASE"
```

6. Tambahkan baris berikut ke .gitignore agar konfigurasi rahasia dan dependensi tidak masuk ke Git:

```
.env
node_modules/
```

7. Ubah bagian awal app.js menjadi seperti berikut. Pertahankan konfigurasi EJS, middleware formulir, dan seluruh rute yang sudah dibuat:

```
require('dotenv').config();
const express = require('express');
const { body, validationResult } = require('express-validator');
const app = express();
const port = process.env.PORT || 3000;
```

Pemanggilan app.listen(port, ...) tetap berada di bagian akhir app.js. Jangan mencetak DATABASE_URL ke terminal atau menyertakannya dalam tangkapan layar.

Uji hasil

Jalankan ulang node app.js, lalu buka /form untuk memastikan aplikasi tetap bekerja. Pemeriksaan koneksi basis data dilakukan melalui sinkronisasi skema pada Lab 3.

Hasil pembelajaran

Proyek Supabase siap. PORT dan URI Session pooler tersimpan pada .env. dotenv dimuat sebelum konfigurasi aplikasi; koneksi database diuji pada Lab 3.

LAB 03

Menghubungkan aplikasi dengan Prisma ORM

Tujuan

Mendefinisikan model Barang dan menyinkronkannya dengan PostgreSQL Supabase menggunakan Prisma db push.

Langkah praktik

1. Gunakan Prisma 6.19.0 agar konfigurasi dan impor CommonJS pada latihan konsisten. Instal CLI serta Client dengan versi yang sama, lalu inisialisasi Prisma:

```
npm install --save-dev --save-exact prisma@6.19.0
npm install --save-exact @prisma/client@6.19.0
npx prisma init --datasource-provider postgresql
```

2. Buka prisma/schema.prisma dan ganti isinya dengan skema lengkap berikut. Jangan menambahkan blok datasource atau generator kedua:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Barang {
  id       Int      @id @default(autoincrement())
  nama     String
  harga    Int
  stok     Int
  createdAt DateTime @default(now())
}
```

3. Jika inisialisasi membuat prisma.config.ts, ganti isinya dengan konfigurasi berikut. Jika belum ada, buat di folder utama proyek. Impor dotenv memastikan CLI membaca .env:

```
import 'dotenv/config';
import { defineConfig } from 'prisma/config';

export default defineConfig({
  schema: 'prisma/schema.prisma'
});
```

4. Pastikan .env tetap berisi URI Session pooler Supabase port 5432 dengan sslmode=require. Jalankan validasi skema, sinkronisasi tabel, dan pembuatan Prisma Client:

```
npx prisma validate
npx prisma db push
```

Uji hasil

Terminal menyatakan skema valid dan database sinkron. Jalankan npx prisma studio, lalu buka model Barang atau periksa melalui Table Editor Supabase. Tabel masih kosong. Akhiri Studio dengan Ctrl + C.

Hasil pembelajaran

db push membentuk tabel latihan tanpa basis data shadow dan tanpa riwayat migrasi. Jika muncul peringatan kehilangan data, hentikan; jangan gunakan opsi pemaksaan. Proyek produksi memerlukan alur Prisma Migrate tersendiri.

Rujukan konfigurasi Prisma 6: https://www.prisma.io/docs/orm/v6/reference/prisma-config-reference

LAB 04

Operasi CRUD dasar

Tujuan

Membuat halaman manajemen barang dengan Create, Read, Update, dan Delete menggunakan Express, EJS, serta Prisma. Setiap perubahan disimpan pada PostgreSQL Supabase dan diperiksa kembali melalui daftar barang.

Persiapan dan urutan kode

Selesaikan Lab 1 sampai Lab 3. Model Barang memiliki kolom id, nama, harga, stok, dan createdAt. Pastikan app.js memuat dotenv, impor body serta validationResult, pengaturan EJS, dan middleware express.urlencoded sebelum rute.

Tambahkan seluruh kode lab ini sebelum app.listen(...). Jika rute /barang dari percobaan sebelumnya sudah ada, ganti dengan rute dalam lab ini. Gunakan satu instansi Prisma Client dan satu penanganan galat setelah semua rute.

Di dekat bagian impor app.js, tambahkan Prisma Client:

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
```

Tambahkan pemeriksaan data dan ID berikut setelah konfigurasi middleware, sebelum rute barang:

```
const validasiBarang = [
 body('nama').isString().bail().trim().notEmpty(),
 body('harga').isString().bail()
  .isInt({ min: 0, max: 2147483647 }),
 body('stok').isString().bail()
  .isInt({ min: 0, max: 2147483647 })
];

function cekDataBarang(req, res, next) {
 if (!validationResult(req).isEmpty()) {
  return res.status(422).type('text/plain').send(
   'Data tidak valid. Isi nama, harga, dan stok dengan benar.'
  );
 }
 next();
}

function cekIdBarang(req, res, next) {
 const id = Number(req.params.id);
 if (!/^[0-9]+$/.test(req.params.id) ||
    !Number.isInteger(id) || id < 1 || id > 2147483647) {
  return res.status(400).type('text/plain').send(
   'ID barang tidak valid.'
  );
 }
 req.barangId = id;
 next();
}
```

Nama wajib berupa teks yang tidak kosong. Harga dan stok wajib berupa bilangan bulat dari 0 sampai 2147483647. Data diperiksa sebelum Number() dipanggil. ID harus berupa angka bulat positif dalam batas Int; nilai seperti abc, 1abc, 0, atau -1 ditolak dengan status 400.

Read dan daftar barang

Buat views/barang.ejs dengan kode lengkap berikut. Kolom Aksi menyediakan tautan Edit dan formulir Hapus untuk setiap ID:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Manajemen barang</title>
 <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body class="container mt-5">
 <h2>Manajemen barang</h2>
 <a href="/barang/tambah" class="btn btn-primary mb-3">Tambah barang</a>
 <div class="table-responsive">
  <table class="table table-bordered">
   <thead>
    <tr><th>ID</th><th>Nama</th><th>Harga</th><th>Stok</th><th>Aksi</th></tr>
   </thead>
   <tbody>
    <% if (listBarang.length === 0) { %>
     <tr><td colspan="5">Belum ada barang.</td></tr>
    <% } %>
    <% listBarang.forEach(b => { %>
     <tr>
      <td><%= b.id %></td>
      <td><%= b.nama %></td>
      <td><%= b.harga %></td>
      <td><%= b.stok %></td>
      <td>
       <a href="/barang/edit/<%= b.id %>"
        class="btn btn-warning btn-sm">Edit</a>
       <form action="/barang/hapus/<%= b.id %>" method="POST"
        class="d-inline" onsubmit="return confirm('Yakin hapus barang ini?')">
        <button type="submit" class="btn btn-danger btn-sm">Hapus</button>
       </form>
      </td>
     </tr>
    <% }); %>
   </tbody>
  </table>
 </div>
</body>
</html>
```

Gunakan <%= ... %> untuk menampilkan nilai barang agar EJS meng-escape karakter HTML. Konfirmasi Hapus membatalkan pengiriman jika pengguna memilih Batal. Hapus memakai POST, sedangkan GET hanya menampilkan halaman.

Tambahkan rute daftar dan formulir tambah pada app.js:

```
app.get('/barang', async (req, res, next) => {
 try {
  const barangs = await prisma.barang.findMany({ orderBy: { id: 'asc' } });
  res.render('barang', { listBarang: barangs });
 } catch (err) {
  next(err);
 }
});
app.get('/barang/tambah', (req, res) => {
 res.render('tambah-barang');
});
```

findMany() membaca daftar yang diurutkan berdasarkan ID. Data dikirim ke EJS sebagai listBarang; tabel kosong menampilkan pesan Belum ada barang.

Create dan formulir tambah barang

Buat views/tambah-barang.ejs dengan kode lengkap berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Tambah barang</title>
 <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body class="container mt-5">
 <h2>Tambah barang</h2>
 <form action="/barang/tambah" method="POST">
  <label for="nama" class="form-label">Nama barang</label>
  <input id="nama" type="text" name="nama" required
   class="form-control mb-2">
  <label for="harga" class="form-label">Harga dalam rupiah</label>
  <input id="harga" type="number" name="harga"
   min="0" max="2147483647" step="1" required
   class="form-control mb-2">
  <label for="stok" class="form-label">Stok</label>
  <input id="stok" type="number" name="stok"
   min="0" max="2147483647" step="1" required
   class="form-control mb-2">
  <button type="submit" class="btn btn-success">Simpan</button>
  <a href="/barang" class="btn btn-secondary">Batal</a>
 </form>
</body>
</html>
```

Tambahkan rute penyimpanan. validasiBarang dan cekDataBarang adalah pemeriksaan bersama yang dibuat pada bagian persiapan:

```
app.post('/barang/tambah', validasiBarang, cekDataBarang,
 async (req, res, next) => {
  try {
   const { nama, harga, stok } = req.body;
   await prisma.barang.create({
    data: { nama, harga: Number(harga), stok: Number(stok) }
   });
   res.redirect('/barang');
  } catch (err) {
   next(err);
  }
 }
);
```

Jika data tidak valid, server mengirim status 422 tanpa menyimpan barang. Gunakan tombol Kembali pada peramban untuk memperbaiki isian. Setelah berhasil, res.redirect mengarahkan pengguna ke /barang untuk membaca hasil terbaru.

Update dan formulir edit barang

Tambahkan rute GET berikut. cekIdBarang memvalidasi parameter URL, lalu findUnique() mengambil satu barang. ID yang valid tetapi tidak ditemukan menghasilkan status 404:

```
app.get('/barang/edit/:id', cekIdBarang,
 async (req, res, next) => {
  try {
   const barang = await prisma.barang.findUnique({
    where: { id: req.barangId }
   });
   if (!barang) {
    return res.status(404).type('text/plain').send(
     'Barang tidak ditemukan.'
    );
   }
   res.render('edit-barang', { b: barang });
  } catch (err) {
   next(err);
  }
 }
);
```

Buat views/edit-barang.ejs dengan kode lengkap berikut. Nilai lama diisikan melalui value; ID pada action menentukan barang yang diperbarui:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Edit barang</title>
 <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body class="container mt-5">
 <h2>Edit barang</h2>
 <form action="/barang/edit/<%= b.id %>" method="POST">
  <label for="nama" class="form-label">Nama barang</label>
  <input id="nama" type="text" name="nama" required
   class="form-control mb-2" value="<%= b.nama %>">
  <label for="harga" class="form-label">Harga dalam rupiah</label>
  <input id="harga" type="number" name="harga"
   min="0" max="2147483647" step="1" required
   class="form-control mb-2" value="<%= b.harga %>">
  <label for="stok" class="form-label">Stok</label>
  <input id="stok" type="number" name="stok"
   min="0" max="2147483647" step="1" required
   class="form-control mb-2" value="<%= b.stok %>">
  <button type="submit" class="btn btn-success">Simpan perubahan</button>
  <a href="/barang" class="btn btn-secondary">Batal</a>
 </form>
</body>
</html>
```

Menyimpan perubahan barang

Tambahkan rute POST untuk menyimpan perubahan nama, harga, dan stok. Gunakan pemeriksaan data yang sama dengan fitur tambah:

```
app.post('/barang/edit/:id', cekIdBarang,
 validasiBarang, cekDataBarang, async (req, res, next) => {
  try {
   const { nama, harga, stok } = req.body;
   await prisma.barang.update({
    where: { id: req.barangId },
    data: { nama, harga: Number(harga), stok: Number(stok) }
   });
   res.redirect('/barang');
  } catch (err) {
   next(err);
  }
 }
);
```

update() hanya mengubah barang yang cocok dengan where: { id: req.barangId }. ID dan jumlah baris tetap sama. Tombol Batal kembali ke daftar tanpa mengirim formulir atau menyimpan perubahan.

Delete dan penanganan galat

Tambahkan rute POST berikut untuk menerima formulir Hapus dari tabel barang:

```
app.post('/barang/hapus/:id', cekIdBarang,
 async (req, res, next) => {
  try {
   await prisma.barang.delete({
    where: { id: req.barangId }
   });
   res.redirect('/barang');
  } catch (err) {
   next(err);
  }
 }
);
```

delete() menghapus satu barang berdasarkan ID. Konfirmasi di peramban membantu mencegah salah klik; server tetap memvalidasi ID pada setiap permintaan. Operasi tidak dijalankan melalui tautan GET.

Setelah semua rute dan sebelum app.listen(...), tambahkan penanganan galat ini. Jika sudah ada penanganan galat dari percobaan sebelumnya, ganti dengan kode berikut:

```
app.use((err, req, res, next) => {
 if (err.code === 'P2025') {
  return res.status(404).type('text/plain').send(
   'Barang tidak ditemukan atau sudah dihapus.'
  );
 }
 console.error(err.message);
 res.status(500).type('text/plain').send(
  'Operasi gagal. Periksa koneksi dan pesan pada terminal.'
 );
});
```

Prisma mengirim kode P2025 ketika sasaran update() atau delete() sudah tidak ada, misalnya dihapus dari tab lain. Aplikasi membalas 404. Kegagalan lain diteruskan sebagai 500; detailnya diperiksa melalui terminal.

Uji hasil

1. Jalankan ulang node app.js dan buka http://localhost:3000/barang. Jika tabel masih kosong, pesan Belum ada barang harus muncul tanpa galat.

2. Create dan Read: tambahkan Buku Tulis dengan harga 12000 dan stok 25, lalu Pulpen dengan harga 5000 dan stok 10. Catat kedua ID. Daftar menampilkan dua baris baru dengan nilai yang sesuai.

3. Update: klik Edit pada Buku Tulis. Pastikan tiga isian berisi nilai lama. Ubah nama menjadi Buku Tulis A5, harga menjadi 15000, dan stok menjadi 20, lalu klik Simpan perubahan. ID tetap sama dan jumlah baris tidak bertambah. Buka Edit sekali lagi untuk memeriksa nilai terbaru.

4. Batal edit: ubah satu isian pada formulir edit, lalu klik Batal. Nilai dalam daftar harus tetap seperti hasil penyimpanan terakhir.

5. Validasi: pada formulir tambah dan edit, gunakan alat pengembang peramban untuk menambahkan atribut novalidate pada tag form. Kirim nama berupa spasi, harga -1, stok 1.5, atau angka 2147483648. Server harus mengirim 422, tanpa menambah atau mengubah data. Harga dan stok 0 harus diterima. Pulihkan formulir dengan memuat ulang halaman.

6. Batal hapus: klik Hapus pada Pulpen, lalu pilih Batal pada dialog. Baris tetap ada. Klik Hapus lagi dan setujui; hanya Pulpen yang hilang, sedangkan Buku Tulis A5 tetap ada.

7. ID dan barang hilang: buka /barang/edit/abc, /barang/edit/1abc, dan /barang/edit/0 untuk memeriksa status 400. Buka /barang/edit/ID_PULPEN dengan mengganti ID_PULPEN menggunakan ID yang baru dihapus; hasilnya 404. Gunakan tab Network pada alat pengembang peramban untuk melihat status HTTP.

8. Sasaran hilang saat edit: buka formulir edit Buku Tulis A5 pada satu tab. Hapus barang itu dari tab daftar lainnya, lalu kirim perubahan dari tab edit. Server harus mengirim 404. Kirim ulang POST hapus untuk ID yang sama melalui fitur pengiriman ulang pada tab Network; hasilnya juga 404.

9. Penyimpanan: tambahkan kembali dua barang latihan, ubah salah satunya, lalu hentikan dan jalankan ulang server. Daftar harus tetap menampilkan data terakhir. Barang yang dihapus tidak muncul kembali.

Hasil pembelajaran

create() menambah barang, findMany() membaca daftar, findUnique() mengisi formulir edit, update() menyimpan perubahan, dan delete() menghapus barang. Validasi data, pemeriksaan ID, serta respons 400, 404, 422, dan 500 membantu memeriksa setiap alur CRUD.

Rujukan CRUD Prisma 6: https://docs.prisma.io/docs/orm/v6/prisma-client/queries/crud

LAB 05

Komponen partial EJS

Tujuan

Memisahkan struktur header dan footer agar beberapa halaman menggunakan komponen yang sama tanpa menyalin seluruh susunan HTML.

Langkah praktik

1. Buat folder views/partials jika belum ada. Pertahankan navbar.ejs dari Hari 1 untuk halaman lama. Pada lab ini, gunakan header dan footer baru untuk halaman barang.
2. Buat views/partials/header.ejs dengan isi berikut. Navigasi memakai flex-wrap agar tautan tetap terlihat pada layar sempit:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Aplikasi Node Modern</title>
 <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
</head>
<body>
 <nav class="navbar navbar-dark bg-dark mb-4" aria-label="Navigasi utama">
  <div class="container flex-wrap gap-2">
   <a class="navbar-brand" href="/">MyApp</a>
   <div class="navbar-nav flex-row gap-3 ms-auto">
    <a class="nav-link" href="/form">Formulir</a>
    <a class="nav-link" href="/barang">Barang</a>
   </div>
  </div>
 </nav>
```

3. Buat views/partials/footer.ejs dengan isi berikut:

```
 <footer class="text-center mt-5 p-3 bg-light">
  <p class="mb-0">
   &copy; <%= new Date().getFullYear() %> Aplikasi Modern.
   Hak cipta dilindungi.
  </p>
 </footer>
 <script
  src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
 </script>
</body>
</html>
```

Header membuka dokumen HTML, memuat Bootstrap, dan menampilkan navigasi. Footer menutup dokumen dan memuat JavaScript Bootstrap. Kedua partial dipanggil satu kali pada setiap halaman yang menggunakannya.

Menerapkan partial pada halaman barang

4. Buka views/barang.ejs. Hapus bagian awal dari <!DOCTYPE html> sampai tag pembuka <body ...>, lalu ganti dengan dua baris berikut:

```
<%- include('partials/header') %>
<main class="container">
```

5. Pertahankan seluruh isi halaman mulai dari <h2>Manajemen barang</h2> sampai penutup yang membungkus tabel. Hapus tag penutup </body> dan </html>, lalu ganti dengan:

```
</main>
<%- include('partials/footer') %>
```

6. Lakukan penggantian yang sama pada views/tambah-barang.ejs dan views/edit-barang.ejs: ganti bagian awal sampai pembuka body dengan include header dan pembuka main. Pertahankan judul, action, value, dan seluruh form. Ganti penutup body dan html dengan penutup main serta include footer.

Susunan setiap halaman kini terdiri atas include header, <main class="container">, konten halaman, </main>, lalu include footer. Jangan menyisakan tag html, head, atau body tambahan di halaman pemanggil.

7. Simpan berkas dan buka /barang, /barang/tambah, serta halaman Edit dari salah satu baris barang. Jika server memakai Nodemon, perubahan EJS dapat diperiksa dengan memuat ulang halaman. Jika diperlukan, jalankan ulang node app.js.

Uji hasil

Pastikan ketiga halaman memiliki navigasi dan footer yang sama. Tautan Barang harus menuju /barang, tautan Formulir menuju /form, dan merek MyApp menuju halaman utama dari Hari 1.

Ubah teks MyApp menjadi Inventaris Inixindo hanya pada views/partials/header.ejs. Muat ulang ketiga halaman dan periksa nama baru. Tambahkan barang, edit nilainya, lalu hapus barang uji untuk memastikan seluruh alur CRUD tetap berfungsi setelah pemisahan HTML.

Gunakan <%- include(...) %> untuk memasukkan HTML partial. Untuk menampilkan nilai data, tetap gunakan <%= ... %> seperti pada nama barang dan pesan kesalahan.

Hasil pembelajaran

Halaman daftar, tambah, dan edit barang berbagi header serta footer. Perubahan navigasi cukup dilakukan pada satu berkas partial, sementara rute dan pengolahan data tetap berada di app.js.

Pemeriksaan akhir Hari 2

Pastikan formulir menolak input tidak valid, operasi tambah, baca, edit, dan hapus berjalan sesuai ID, serta dua data terakhir tetap tersimpan setelah server dijalankan ulang. Periksa navigasi dan footer pada ketiga halaman barang. Simpan proyek serta package-lock.json; rahasiakan .env.

Pemecahan masalah praktik

Formulir tidak terbuka atau req body kosong

Untuk pesan Cannot GET /form, periksa ejaan rute dan pastikan rute didefinisikan sebelum app.listen(...). Jika req.body kosong, letakkan app.use(express.urlencoded({ extended: true })); sebelum rute POST. Nama input pada EJS harus sama dengan nama yang dibaca dari req.body.

Prisma tidak membaca DATABASE URL

Pastikan .env berada sejajar dengan package.json dan berisi kunci DATABASE_URL. Periksa impor dotenv/config pada prisma.config.ts. Di aplikasi, require('dotenv').config(); harus berjalan sebelum Prisma Client dibuat. Setelah mengubah konfigurasi, jalankan ulang perintah Prisma atau server.

Koneksi atau sinkronisasi skema gagal

Pastikan proyek Supabase aktif. Gunakan URI Session pooler port 5432 dari Connect, nama pengguna lengkap dengan referensi proyek, serta kata sandi database yang sudah di-encode. Tambahkan sslmode=require. Periksa koneksi internet dan akses port 5432. Lab memakai db push, sehingga tidak memerlukan basis data shadow.

Prisma Client belum tersedia atau versi berbeda

Jalankan npm ls prisma @prisma/client dan pastikan keduanya versi 6.19.0. Setelah menyunting skema, jalankan npx prisma generate lalu jalankan ulang server. Panduan memakai generator prisma-client-js dengan impor require('@prisma/client').

Partial tidak ditemukan atau HTML berganda

Periksa lokasi views/partials/header.ejs dan views/partials/footer.ejs. Dari berkas yang langsung berada di folder views, gunakan path partials/header dan partials/footer. Halaman pemanggil hanya menyimpan konten di antara kedua include; struktur html, head, dan body berada pada partial.

Bahan praktik dan rujukan

Bahan praktik Hari 2: Lab_1_Validasi_Form.md, Lab_2_Setup_Database_Supabase.md, Lab_3_Prisma_ORM.md, Lab_4_CRUD_Operation.md, dan Lab_5_Komponen_Partials.md pada folder Materi_V2/Hari_2.

Supabase, koneksi PostgreSQL: https://supabase.com/docs/guides/database/connecting-to-postgres

Supabase, Prisma: https://supabase.com/docs/guides/database/prisma

Prisma 6, kebutuhan Node.js dan kesesuaian versi paket: https://www.prisma.io/docs/guides/upgrade-prisma-orm/v6

Prisma 6, konfigurasi CLI: https://www.prisma.io/docs/orm/v6/reference/prisma-config-reference